

Data Analytics in Cloud Computing

Exercise 2

Virtualization & Containers Lab

IaaS (Azure VM) • Containers (Podman) • Managed containers (ACI/AKS/Container Apps)



Introduction

In this lab you will run the same small analytics workload first on a virtual machine (VM) and then inside a container. The goal is to experience practical differences (kernel sharing, isolation boundaries, portability, and operational responsibility) rather than just learning definitions.

How to copy code (important)

One-liner commands in this document are safe to copy and run.

For longer code blocks (Python files and Containerfile), copy from the repository files to avoid indentation/formatting issues when copying from PDF.

Repo files used in this lab:

- EX_2/wine_metrics.py
- EX_2/Containerfile

Prerequisites

- An Azure subscription with permission to create resources in at least one allowed region.
- An SSH client to connect to the VM.

- Basic Linux command line skills.
- Podman installed either locally or on the VM (this lab shows installation on the VM).

Azure offerings overview (VMs and containers)

VM / virtualization offerings in Azure:

- **Azure Virtual Machines (VMs)** — general purpose IaaS compute; you manage OS patching and installed software.
- **Virtual Machine Scale Sets (VMSS)** — manage a group of load-balanced autoscaling VMs.
- **Azure Spot Virtual Machines** — discounted capacity that can be evicted; good for interruptible batch workloads.
- **Azure Dedicated Host** — dedicated physical server(s) to run your VMs for isolation/compliance.
- **Azure VMware Solution** — run VMware SDDC (vSphere/vSAN/NSX) on dedicated bare-metal in Azure.

Container offerings in Azure:

- **Azure Kubernetes Service (AKS)** — managed Kubernetes for orchestrating containers at scale.
- **Azure Container Instances (ACI)** — run containers on demand without managing VMs/clusters
- **Azure Container Apps** — “serverless containers” for microservices without Kubernetes complexity.
- **Azure App Service (Custom Containers)** — run a containerized web app on managed PaaS hosting.
- **Azure Red Hat OpenShift (ARO)** — managed OpenShift (Kubernetes platform) jointly supported by MS/Red Hat.
- **Azure Container Registry (ACR)** — private registry to store/manage container images.

[Azure Virtual Machines \(overview\)](#)

[Azure Container Apps](#)

[Virtual Machine Scale Sets](#)

[Azure Container Registry \(ACR\)](#)

[Azure Spot Virtual Machines](#)

[App Service \(custom containers\)](#)

[Azure Container Instances \(ACI\)](#)

[Azure Red Hat OpenShift \(ARO\)](#)

[Azure Kubernetes Service \(AKS\)](#)

Dataset (public)

We use the UCI Wine Quality dataset (ID 186). Download happens during the lab.

Dataset page: [UCI Wine Quality](#)

Metrics computed:

- Total rows
- Average alcohol
- High-quality count (quality $\geq$ QUALITY_THRESHOLD)
- Most common quality score

Lab parameter:

- QUALITY_THRESHOLD (default 7). Change it without changing code.

Task 1 — Virtualization (IaaS): Run the job on an Azure VM

Learning objectives

- Provision an Ubuntu VM in Azure.
- Connect via SSH.
- Download the dataset and run the Python script on the VM host OS.

[VM quickstart \(Azure Portal\)](#)

Step 1: Create the VM

In the Azure portal create a Resource Group (e.g., rg-ex2-virt-containers) and then create a Linux VM with:

- Image: Ubuntu Server 24.04 LTS (or similar)
- Size: small B-series for demo
- Authentication: SSH key
- Inbound: allow SSH (22) and restrict source to your IP if possible

Step 2: Connect via SSH

Open the VM in the portal → Connect → SSH, and run the suggested command from your terminal (or Azure Cloud Shell).

Step 3: Download dataset and run the job

Use the one-liner commands below to download the dataset and run the job on the VM:

```
mkdir -p ~/ex2 && cd ~/ex2
sudo apt-get update && sudo apt-get install -y curl unzip
curl -L -o wine_quality.zip "https://archive.ics.uci.edu/static/public/186/wine%2Bquality.zip"
unzip -j -o wine_quality.zip winequality-red.csv -d .
head -n 3 winequality-red.csv

# Copy wine_metrics.py from the repo:
curl -L "https://raw.githubusercontent.com/MySlav/Data-Analytics-in-Cloud-Computing-2025-2026/refs/heads/main/EX_2/wine_metrics.py" -o wine_metrics.py
python3 wine_metrics.py
QUALITY_THRESHOLD=8 python3 wine_metrics.py
```

Task 2 — Containers: Run the same job in a Podman container

Learning objectives

- Install Podman on the VM (or use local Podman).
- Build a container image using Containerfile.
- Run the container with environment variables and a volume mount.

Step 1: Install Podman on Ubuntu

```
sudo apt-get update
sudo apt-get install -y podman
podman --version
```

Step 2: Build the image

Copy the Containerfile from the repo (EX_2/Containerfile) and build:

```
curl -L "https://raw.githubusercontent.com/MySlav/Data-Analytics-in-Cloud-Computing-2025-2026/refs/heads/main/EX_2/Containerfile" -o Containerfile
podman build -t wine-metrics:1 -f Containerfile .
```

Step 3: Run the container (mount the dataset from the host)

```
podman run --rm \
  -e DATA_FILE=/data/winequality-red.csv \
  -e QUALITY_THRESHOLD=7 \
  -v "$PWD/winequality-red.csv:/data/winequality-red.csv:ro" \
  wine-metrics:1
```

Change the threshold without changing code:

```
podman run --rm \
  -e DATA_FILE=/data/winequality-red.csv \
  -e QUALITY_THRESHOLD=8 \
  -v "$PWD/winequality-red.csv:/data/winequality-red.csv:ro" \
  wine-metrics:1
```

Task 3 — Prove key differences (VM vs container)

Use the one-liner commands below to collect evidence for VM vs container differences:

How to interpret the outputs (read this while running the commands):

1) Kernel sharing (host vs container)

Run: `uname -r` on the VM host and inside the container.

Typical output: the kernel version string is identical in both places.

Meaning: containers share the host kernel. A VM runs its own OS (including its own kernel) inside a virtualized machine.

2) Userland differences (/etc/os-release)

Run: `head -n 5 /etc/os-release` on the host and inside the container.

Typical output: the host reports Ubuntu, while the container reports the base image distribution (often Debian for `python:slim`).

Meaning: the container packages a different userland (libraries/tools) but still uses the host kernel.

3) Process isolation (PID namespace)

Run: `podman run --rm <image> ps -ef`.

Typical output: only a small set of processes is visible; the main process inside the container acts like PID 1 in that namespace.

Meaning: containers isolate processes using namespaces; they do not see the full host process tree like a VM OS does.

4) Filesystem behavior (ephemeral vs volume)

Create a file inside the container, then start a new container and check if the file still exists. Next, mount a host folder with `-v` and write output there.

Typical output: the file created inside the container disappears on the next run; the file written to the mounted folder persists on the host.

Meaning: container filesystems are ephemeral by default (image + temporary writable layer). Persistence is explicit via volumes/mounts.

5) Resource limits (per-container constraints)

Run: `podman run --rm --cpus 0.5 --memory 128m <image>`.

Typical output: the job still completes (it is small), but it is constrained by the limits.

Meaning: containers can be constrained per workload; VM sizing constrains the whole machine.

```
# Kernel sharing (host vs container)
uname -r
podman run --rm wine-metrics:1 uname -r

# Userland differences
head -n 5 /etc/os-release
podman run --rm wine-metrics:1 sh -lc 'head -n 5 /etc/os-release'

# Process isolation (PID 1 inside container)
podman run --rm wine-metrics:1 ps -ef

# Ephemeral filesystem vs volume mount
podman run --rm wine-metrics:1 sh -lc 'echo hello > /app/tmp.txt && ls -l /app/tmp.txt'
podman run --rm wine-metrics:1 sh -lc 'ls -l /app/tmp.txt || echo "tmp.txt not found (ephemeral)'"
mkdir -p output
podman run --rm \
-e DATA_FILE=/data/winequality-red.csv \
-e QUALITY_THRESHOLD=7 \
-v "$PWD/winequality-red.csv:/data/winequality-red.csv:ro" \
-v "$PWD/output:/output" \
wine-metrics:1 sh -lc 'python /app/wine_metrics.py > /output/results.json'

# Resource limits
podman run --rm \
--cpus 0.5 --memory 128m \
-e DATA_FILE=/data/winequality-red.csv \
-e QUALITY_THRESHOLD=7 \
-v "$PWD/winequality-red.csv:/data/winequality-red.csv:ro" \
wine-metrics:1
```

Task 4 — Managed container on Azure (ACI)

Goal: Run the same container without managing a VM.

Approach: Build locally on the VM (Podman), push to ACR, run on ACI and view logs.

Why: `az acr build` uses ACR Tasks, which may be blocked in student subscriptions so we can't use Azure Cloud Shell for this.

Recommended execution environment: Azure CLI in the previously built VM

Reference links: [Azure Cloud Shell](#) | [ACR Tasks \(az acr build\)](#) | [ACI logs](#)

Azure Cloud Shell one-liners (build in ACR, run in ACI):

```
# Setup
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
az login
az account show
RG=rg-ex2-virt-containers
LOCATION=<allowed-region> # e.g., francecentral
ACR_NAME=<globally-unique-acr> # must be globally unique
IMAGE_NAME=wine-metrics
IMAGE_TAG=1
ACI_NAME=ex2-wine-metrics
az group create -n "$RG" -l "$LOCATION"

# Register providers
az provider register --namespace Microsoft.ContainerRegistry
az provider register --namespace Microsoft.ContainerInstance

# Check
az provider show --namespace Microsoft.ContainerRegistry --query registrationState -o tsv
az provider show --namespace Microsoft.ContainerInstance --query registrationState -o tsv

# Create ACR
az acr create -g $RG -n $ACR_NAME --sku Basic
LOGIN_SERVER=$(az acr show -n $ACR_NAME --query loginServer -o tsv)

# Enable ACR admin user (lab-friendly way for ACI to pull the image)
az acr update -n $ACR_NAME --admin-enabled true
ACR_USER=$(az acr credential show -n $ACR_NAME --query username -o tsv)
ACR_PASS=$(az acr credential show -n $ACR_NAME --query passwords[0].value -o tsv)

# Build image locally
podman build -t "${IMAGE_NAME}:${IMAGE_TAG}" -f Containerfile .
# Login to ACR with Podman
podman login "$LOGIN_SERVER" -u "$ACR_USER" -p "$ACR_PASS"

# Tag + push
podman tag "${IMAGE_NAME}:${IMAGE_TAG}" "$LOGIN_SERVER/${IMAGE_NAME}:${IMAGE_TAG}"
podman push "$LOGIN_SERVER/${IMAGE_NAME}:${IMAGE_TAG}"
# Run in Azure Container Instances (ACI) and view logs
IMAGE_REF="$LOGIN_SERVER/${IMAGE_NAME}:${IMAGE_TAG}"
az container create -g "$RG" -n "$ACI_NAME" \
  --image "$IMAGE_REF" \
  --os-type Linux \
  --cpu 1 --memory 1 \
  --restart-policy Never \
  --registry-login-server "$LOGIN_SERVER" \
  --registry-username "$ACR_USER" \
  --registry-password "$ACR_PASS" \
  --environment-variables QUALITY_THRESHOLD=8 \
  --command-line "python -c \"import os,urllib.request,zipfile,io,runpy;
url='https://archive.ics.uci.edu/static/public/186/wine%2Bquality.zip'; data=urllib.request.urlopen(url).read();
z=zipfile.ZipFile(io.BytesIO(data)); z.extract('winequality-red.csv','/tmp');
os.environ['DATA_FILE']=' /tmp/winequality-red.csv'; runpy.run_path('/app/wine_metrics.py',
run_name='__main__')\""

az container logs -g $RG -n $ACI_NAME
```

Cleanup

Delete the resource group to avoid costs:

```
az group delete -n rg-ex2-virt-containers --yes --no-wait
```